AI Smart Civic Services

Problem-Solving Hackathon

Python • Artificial Intelligence • Statistics • OOP • Web/Mobile • Deployment

PROJECT TYPE Real-world civic problem-solving hackathon	CORE DOMAIN Civic complaints & local service management
MANDATORY Every participant must implement a genuine AI feature	FINAL OUTCOME Working, deployed civic-service application

Project principle: Build one intelligent platform that allows citizens to report local problems and enables service teams to understand, classify, prioritize and track those problems.

Designed for multiple learning batches with the same real-world problem and different technical benchmarks.

1. Project Overview

AI Smart Civic Services is an end-to-end civic complaint and service-management application. Citizens submit local-service complaints, while the system uses AI to understand the complaint and support service teams in classification, prioritization and management.

The project connects classroom learning with a practical software product. Students demonstrate programming, problem-solving, AI integration, data analysis, system architecture, interface design and deployment.

Core objective

Build a working platform where a citizen can submit a civic complaint and the system can intelligently analyze it, store the result, and make the information useful to administrators or service teams.

Typical civic problems

- Broken streetlights
- Overflowing garbage bins
- Damaged roads
- Water leakage
- Drainage problems
- Unsafe public areas
- Electricity-related issues
- Other local service problems

2. Real-World Problem

Citizens regularly face local infrastructure and service problems. Reporting can be fragmented, while service teams may struggle to determine which complaints are urgent and which department should handle them.

The hackathon challenge is to create one intelligent civic-service platform that turns an unstructured citizen complaint into structured, actionable information.

3. Example User Journey

A citizen reports:

> **“There is a large water leak near the main road and traffic is becoming difficult.”**

The application should follow a flow similar to:

Step	System behavior
1. Submit	Citizen enters complaint text and optionally uploads an image.
2. Analyze	AI analyzes the complaint content.
3. Classify	Predict category such as Water/Drainage.
4. Prioritize	Estimate Low, Medium, High or Critical priority.
5. Summarize	Generate a short actionable summary for the service team.
6. Store	Save complaint, location, date, status, category, priority and AI output.
7. Manage	Administrator can view, filter, assign and update complaints.
8. Analyze	Dashboard displays statistics, distributions and trends.

4. Mandatory AI Requirement

AI is a mandatory core requirement. A normal CRUD application without a functional AI component does not satisfy the project requirement.

Every participant must implement at least one genuine AI feature that contributes to solving the civic problem.

Acceptable AI capabilities

AI Feature	Example	Role
Complaint Classification	Road / Water / Waste / Electricity / Drainage / Safety / Other	Turns unstructured complaints into structured categories.
Priority Prediction	Low / Medium / High / Critical	Estimates urgency from complaint text and/or available features.
AI Summarization	Short actionable service-team summary	Converts long complaints into concise operational information.
AI Assistant	Ask questions about complaints or system data	Optional natural-language interface.
AI Vision	Analyze uploaded image for visible damage/waste	Optional image-based civic problem analysis.

Teams may use a trained ML model, pretrained NLP model, LLM/API or another explainable AI technique.

Students must explain what the AI receives, what it does, what it returns and its limitations.

5. Suggested Data Model

Students should create a data model capable of representing the complaint and its AI-generated information.

Field	Description
complaint_id	Unique complaint identifier
description	Citizen's complaint text
category	AI/system classification
priority	AI/system priority level
location	Complaint location
date	Submission date/time
status	Open / Assigned / In Progress / Resolved or equivalent
assigned_department	Responsible service department
ai_output	Stored classification, priority, summary or other AI results

6. Complete Step-by-Step Task

Step 1 — Define the Civic Problem

Choose one local-service problem to emphasize. Define the citizen, service team and exact pain point.

Step 2 — Define AI Input and Output

Specify what information is given to AI and what prediction/generation is returned. Example: complaint text → category + priority + summary.

Step 3 — Design the Data Model

Create fields for complaint ID, description, category, priority, location, date, status, department and AI results.

Step 4 — Choose AI Technology

Select an appropriate model/API and explain why it is suitable. AI must solve a real part of the problem.

Step 5 — Prepare Data

Use a small labeled dataset, public dataset, generated training examples or another defensible source.
Clean and validate it.

Step 6 — Build Python Backend

Implement the core logic using Python. Flask or FastAPI are recommended options.

Step 7 — Implement AI

Connect/train the AI model and return a clear prediction or generated result. Save the AI output with the complaint.

Step 8 — Add OOP Architecture

Use classes for entities/services such as Complaint, User, AIAnalyzer, ComplaintManager, DatabaseManager and NotificationManager.

Step 9 — Add Statistics

Calculate complaint counts, category frequencies, average resolution time, priority distribution and trends.

Step 10 — Build the Interface

Create a web or mobile interface for complaint submission and complaint administration.

Step 11 — Add Search and Filters

Allow searching/filtering by category, priority, status, date, location or department.

Step 12 — Add Error Handling

Handle invalid input, AI/API failure, missing fields, database errors and network failures gracefully.

Step 13 — Test the AI

Test several realistic complaints and report whether predictions/summaries are sensible. Do not claim perfect accuracy.

Step 14 — Deploy

Deploy the Python backend/application publicly on a suitable platform.

Step 15 — Demonstrate

Show live complaint submission, AI processing, stored result, dashboard and deployed application.

7. Batch Benchmarks

The civic problem remains the same across batches. The expected technical depth changes according to learning outcomes. AI remains mandatory for all three.

Batch	Primary Focus	Minimum AI Contribution
1. Advance and Passed-out AI & Data Science	Advanced AI + data integration + deployment. Stronger ML/NLP, analytics, API integration or AI assistant.	AI classification plus priority prediction or summarization. Advanced individuals should implement at least two AI capabilities.
2. Batch 4 — Statistics	Use statistics to understand complaint patterns, priorities, departments, resolution times and trends.	AI classification or priority prediction is mandatory; statistics must explain the resulting data.
3. Batch 5 — OOP	Build the system around classes, objects, methods, encapsulation and modular responsibilities.	AIAnalyzer/AIService must be a clear class integrated into the OOP workflow.

8. Benchmark 1 — Advance and Passed-out AI & Data Science

Challenge: Build an advanced AI-powered civic intelligence platform. The system should not only store complaints but intelligently understand them and support decision-making.

- AI complaint classification is mandatory.
- Add AI priority/severity prediction OR AI summarization.
- Use meaningful data preprocessing and evaluation.
- Provide an analytics dashboard.
- Deploy the application publicly.
- Advanced option: image-based complaint analysis, RAG-based civic assistant, duplicate complaint detection or responsible-department recommendation.

Expected outcome: A production-style prototype demonstrating AI, data processing, backend engineering, user interface and deployment.

9. Benchmark 2 — Batch 4: Statistics

Challenge: Build an AI-powered civic complaint analytics application where statistics are a major part of decision-making.

- AI must classify complaints or predict their priority.
- Calculate mean, median, mode, minimum, maximum, range, variance and standard deviation where relevant.
- Use frequency distributions for complaint categories.
- Use quartiles/IQR and lower/upper fences where meaningful.
- Show charts for categories, priorities, locations or resolution times.
- Explain what the statistics mean rather than displaying numbers only.

Expected outcome: A deployed dashboard that uses AI to organize complaints and statistics to explain what problems are most common, urgent or slow to resolve.

10. Benchmark 3 — Batch 5: OOP

Challenge: Build the same civic-service solution using a strong object-oriented architecture.

- Use classes and objects throughout the core system.
- Create an AIAnalyzer or AIService class responsible for AI operations.
- Create suitable classes such as Complaint, Citizen, Department, ComplaintManager and DatabaseManager.
- Use constructors, attributes, methods, encapsulation and inheritance where appropriate.
- Keep each class responsible for a clear part of the system.
- AI output must become part of the application workflow, not a separate demonstration.

Expected outcome: A deployed OOP-based civic application where a complaint passes through objects/services, reaches the AI component, is stored and appears in the interface.

11. Recommended System Architecture

The following architecture is recommended. Teams may use different technologies where appropriate, but separation of responsibilities should remain clear.

Layer	Recommended implementation
Citizen / Admin UI	HTML/CSS/JavaScript, Streamlit, Flutter, React Native or another suitable interface
Python API	Flask or FastAPI
Complaint Manager	Application/business logic for creating, updating, assigning and tracking complaints
AI Service	scikit-learn, TensorFlow/Keras, PyTorch, Hugging Face or an AI API
Database	SQLite or another appropriate database
Analytics	Complaint counts, frequencies, priority distribution, resolution-time statistics and trends
Deployment	Render, Railway, PythonAnywhere, AWS or another suitable public platform

Reference flow

Citizen/Mobile/Web UI → Python API → Complaint Manager → AI Service → Database → Admin Dashboard

12. Example AI Workflow

Complaint text → preprocessing/model/API → category + priority + summary → stored

AI result → administrator dashboard

13. Submission Requirements

Deliverable	Requirement
GitHub Repository	Source code with a clear project structure.
Public Deployment / APK	Working public URL or APK with installation instructions.
README	Problem, features, architecture, AI technology, setup and usage.
Architecture Diagram	Show where AI is used and how components interact.
AI Testing Evidence	Example inputs, outputs and discussion of limitations.
UI Screenshots	Citizen and administrator interfaces.
Demo Video	3–5 minute demonstration.

14. Final Demonstration Checklist

- Explain the civic problem and target users.
- Submit a live complaint.
- Show AI classification and/or priority/summarization.
- Show the stored complaint and AI output.
- Demonstrate administrator management.
- Demonstrate search and filtering.
- Show statistics and dashboard insights.
- Explain the AI technology and limitations.
- Demonstrate the deployed application.
- Answer technical questions about the implementation.

15. Suggested Hackathon Evaluation Rubric

The project specification defines required capabilities and batch benchmarks but does not specify a numerical judging rubric. The following is a recommended rubric added for practical hackathon evaluation.

Category	Marks	Assessment focus
Problem Understanding	10	Clear civic problem, users, pain point and solution scope
AI Implementation	25	Meaningful AI feature, model/API choice, integration, testing and limitations
Python / Backend	15	Backend logic, API, data handling, error handling and code quality
Statistics / Analytics	15	Relevant metrics, distributions, visualizations and interpretation
OOP / Architecture	10	Classes, responsibilities, modularity and appropriate encapsulation
UI/UX	10	Citizen/admin usability, clarity and workflow
Deployment	10	Working public deployment and reproducibility
Presentation / Demo	5	Live demonstration, explanation and Q&A;
TOTAL	**100**	**Recommended total**

For Batch 4, judges should emphasize Statistics/Analytics. For Batch 5, emphasize OOP/Architecture. For the passed-out AI & Data Science batch, emphasize AI implementation, data integration and deployment.

16. Important Rules

- AI is mandatory: A project without a functional AI component cannot receive full marks.
- AI must contribute to the solution: Simply displaying an AI-generated sentence is not sufficient.
- Explainability: Every individual must explain the AI input, processing/model, output and limitations.
- Security: Do not expose API keys or private credentials in GitHub.
- Responsible data use: Respect data-collection and website/API rules when using external sources.
- Deployment: The final application must be deployed and testable.

17. Technology Options

Component	Examples
Frontend	HTML/CSS/JavaScript, Streamlit, Flutter, React Native
Backend	Python, Flask, FastAPI
Database	SQLite or another appropriate database
AI / ML	scikit-learn, TensorFlow/Keras, PyTorch, Hugging Face, AI APIs
Deployment	Render, Railway, PythonAnywhere, AWS or another suitable platform

18. Final Project Vision

The goal is not to build a generic CRUD complaint system. The goal is to demonstrate how programming, AI, statistics, OOP, interfaces and deployment can work together to solve a real civic problem.

Citizen Problem → AI Understanding → Structured Complaint → Service Management → Statistical Insight → Better Decision

The same civic problem can be used across multiple classes while the technical expectations remain aligned with what each group has learned.

For the advanced AI & Data Science batch, the project should evolve into a production-style AI civic intelligence platform. For the Statistics batch, it should become a statistics-driven civic analytics system.
For the OOP batch, it should become a clean, modular, object-oriented civic application.

End of Project Specification